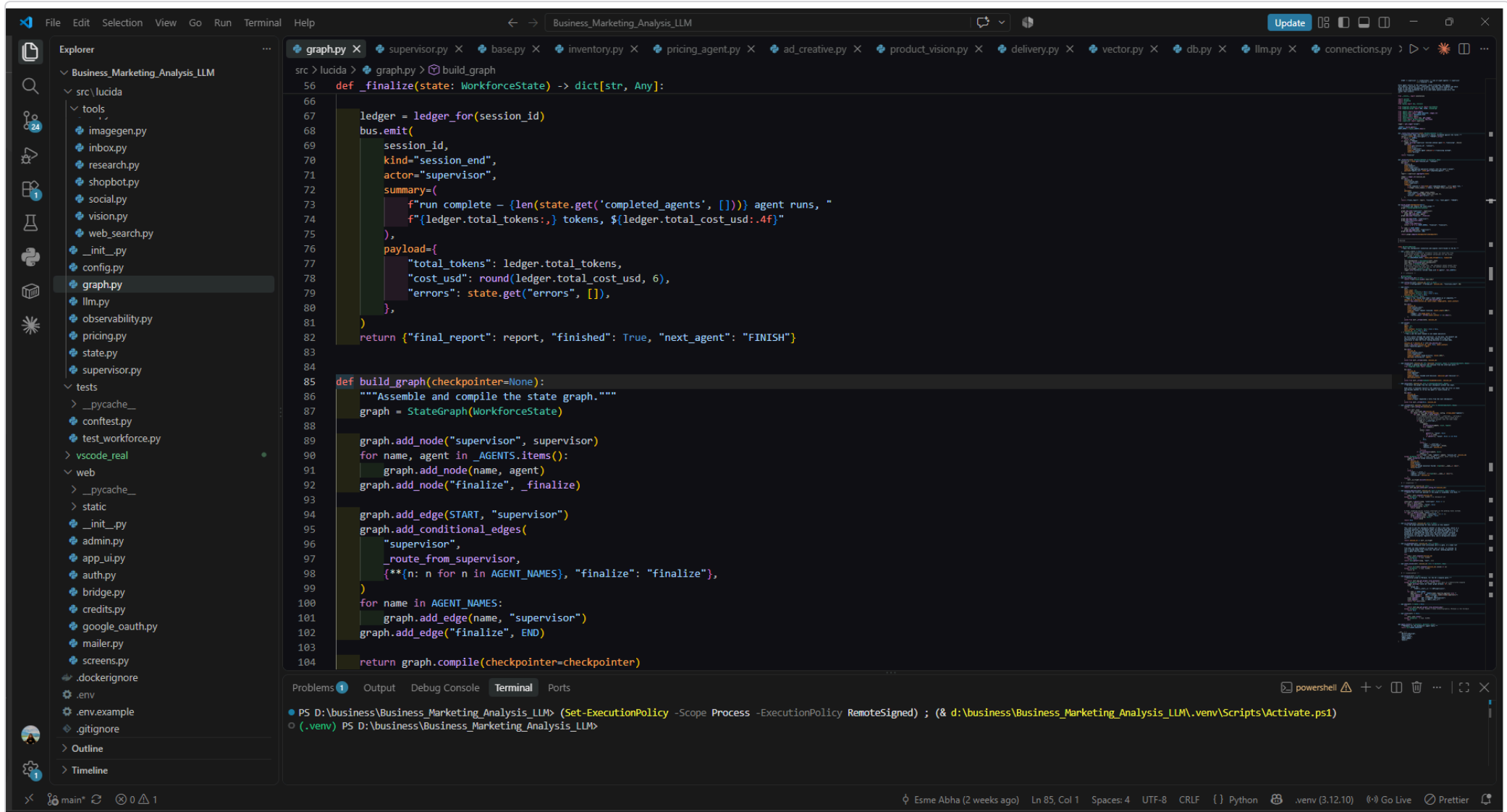


Lucida — code evidence

Esme Moula Chowdhury Abha · 26-94089-2 · sole author

System	LangGraph supervisor and 8 specialists, per-shop SQLite, vector memory
Size	50 Python files · ~21,000 lines · 22 tables · 87 tests
Authorship	88 commits, one contributor, 14 Aug – 3 Sep 2026
Live integrations	Pathao courier (a real consignment booked), Telegram, OpenAI, DuckDuckGo
This document	24 screenshots, each captioned with the criterion it evidences

Each page is an unedited screenshot of the file open in Visual Studio Code, taken at the line named in its caption, so any of them can be opened in the accompanying source and checked.

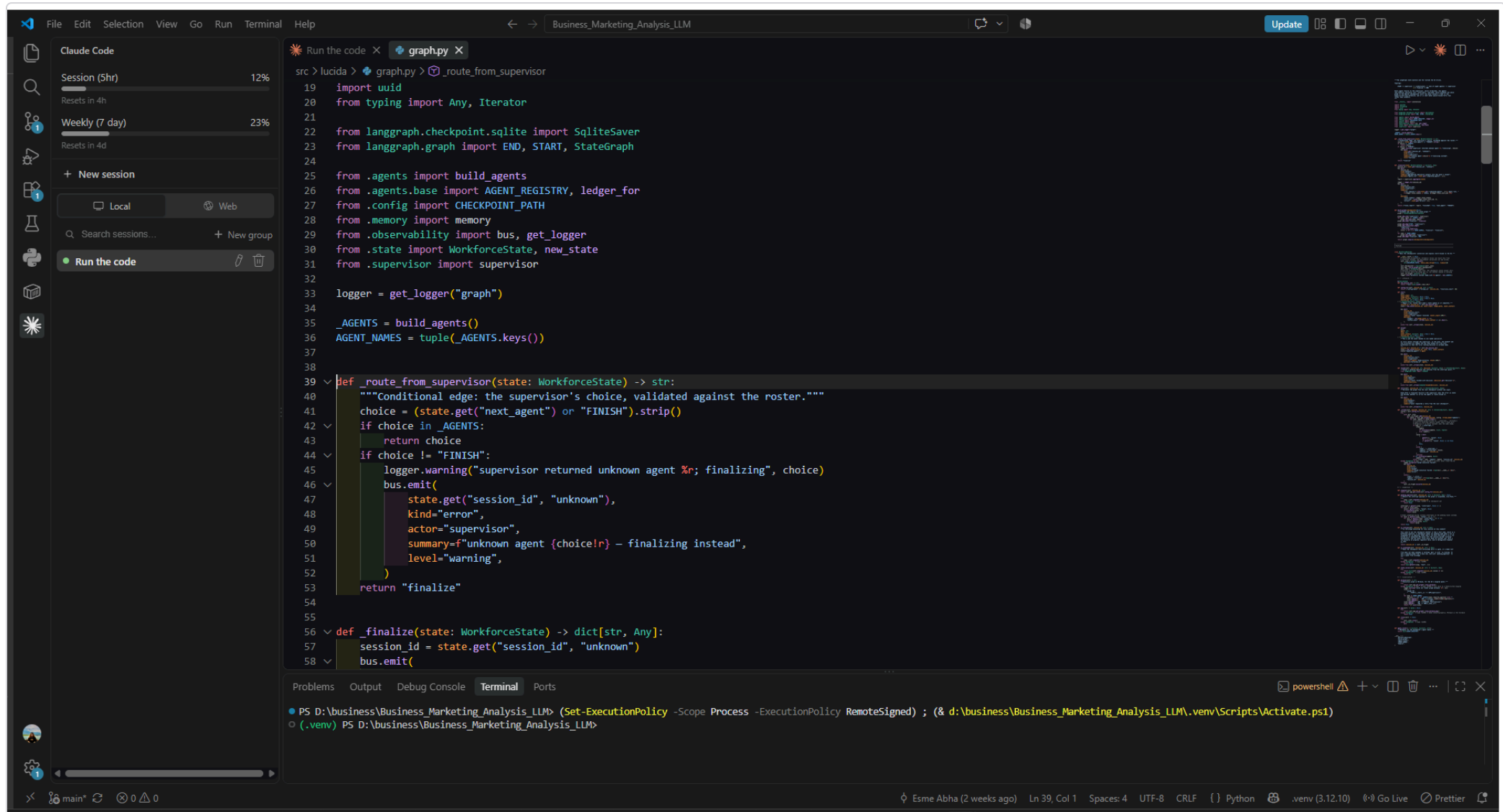


```
src > lucida > graph.py > build_graph
56 def _finalize(state: WorkforceState) -> dict[str, Any]:
66     ledger = ledger_for(session_id)
67     bus.emit(
68         session_id,
69         kind="session_end",
70         actor="supervisor",
71         summary=(
72             f"run complete - {len(state.get('completed_agents', []))} agent runs, "
73             f"[ledger.total_tokens:,{} tokens, ${ledger.total_cost_usd:.4f}"
74         ),
75     ),
76     payload={
77         "total_tokens": ledger.total_tokens,
78         "cost_usd": round(ledger.total_cost_usd, 6),
79         "errors": state.get("errors", []),
80     },
81 )
82 return {"final_report": report, "finished": True, "next_agent": "FINISH"}

85 def build_graph(checkpointer=None):
86     """Assemble and compile the state graph."""
87     graph = StateGraph(WorkforceState)
88
89     graph.add_node("supervisor", supervisor)
90     for name, agent in _AGENTS.items():
91         graph.add_node(name, agent)
92     graph.add_node("finalize", _finalize)
93
94     graph.add_edge(START, "supervisor")
95     graph.add_conditional_edges(
96         "supervisor",
97         _route_from_supervisor,
98         {(n: n for n in AGENT_NAMES), "finalize": "finalize"},
99     )
100     for name in AGENT_NAMES:
101         graph.add_edge(name, "supervisor")
102     graph.add_edge("finalize", END)
103
104     return graph.compile(checkpointer=checkpointer)
```

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

`src/lucida/graph.py` — Every request enters at the supervisor; specialists return to it and never call each other, so the route taken is a decision rather than a fixed pipeline.



```
src > lucida > graph.py > _route_from_supervisor
19 import uuid
20 from typing import Any, Iterator
21
22 from langgraph.checkpoint.sqlite import SqliteSaver
23 from langgraph.graph import END, START, StateGraph
24
25 from .agents import build_agents
26 from .agents.base import AGENT_REGISTRY, ledger_for
27 from .config import CHECKPOINT_PATH
28 from .memory import memory
29 from .observability import bus, get_logger
30 from .state import WorkforceState, new_state
31 from .supervisor import supervisor
32
33 logger = get_logger("graph")
34
35 _AGENTS = build_agents()
36 AGENT_NAMES = tuple(_AGENTS.keys())
37
38
39 def _route_from_supervisor(state: WorkforceState) -> str:
40     """Conditional edge: the supervisor's choice, validated against the roster."""
41     choice = (state.get("next_agent") or "FINISH").strip()
42     if choice in _AGENTS:
43         return choice
44     if choice != "FINISH":
45         logger.warning("supervisor returned unknown agent %r; finalizing", choice)
46         bus.emit(
47             state.get("session_id", "unknown"),
48             kind="error",
49             actor="supervisor",
50             summary=f"unknown agent {choice!r} - finalizing instead",
51             level="warning",
52         )
53     return "finalize"
54
55
56 def _finalize(state: WorkforceState) -> dict[str, Any]:
57     session_id = state.get("session_id", "unknown")
58     bus.emit(
```

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

src/lucida/graph.py — What makes the star a star: the supervisor's choice decides the next node, or ends the run.

The image shows a VS Code editor window with the following content:

Left Sidebar (Claude Code):

- Session (5hr): 12%
- Resets in 4h
- Weekly (7 day): 23%
- Resets in 4d
- + New session
- Local Web
- Search sessions... + New group
- Run the code

Main Editor (supervisor.py):

```

63
64 FINISH - The owner's request has been fully answered. Stop.
65
66
67 PLANNER_SYSTEM = f"""You are the Supervisor of an AI workforce that runs a small business end to end.
68
69 A business owner has sent you a request. Produce a short plan naming which specialist
70 handles which step.
71
72 {AGENT_CATALOG}
73
74 Rules:
75 - Keep the plan to the minimum number of steps that genuinely answers the request. A
76 simple question does not need all eight agents.
77 - Only include product_vision if an image was uploaded.
78 - Only include delivery if the owner is actually asking to ship something.
79 - Put reporting last when the request spans several stages; omit it for narrow requests.
80 - Each step must name the agent responsible.
81
82
83 ROUTER_SYSTEM = f"""You are the Supervisor of an AI workforce that runs a small business end to end.
84
85 Decide which single agent should work NEXT, or FINISH if the owner's request has been
86 fully answered.
87
88 {AGENT_CATALOG}
89
90 Rules:
91 - Choose exactly ONE agent name from the list, or FINISH.
92 - FINISH IMMEDIATELY, before any agent runs, when the message does not need
93 specialist work: a greeting, thanks, an acknowledgement, small talk, or a
94 question about you and what you can do. You will still write the reply
95 yourself, so the owner is answered – they simply do not need eight
96 specialists and a web search to be told hello. Dispatching the workforce
97 at "hi" costs the owner minutes and thousands of tokens for a sentence.
98 - Do NOT re-run an agent that has already completed successfully unless its output was an
99 error, or new information genuinely changes its inputs.
100 - Answer the question that was asked, and stop. Once the agents that were run
101 can answer it, choose FINISH – do not add adjacent work the owner did not
102 ask for. "Which product should I reorder first?" is answered by stock
  
```

Terminal:

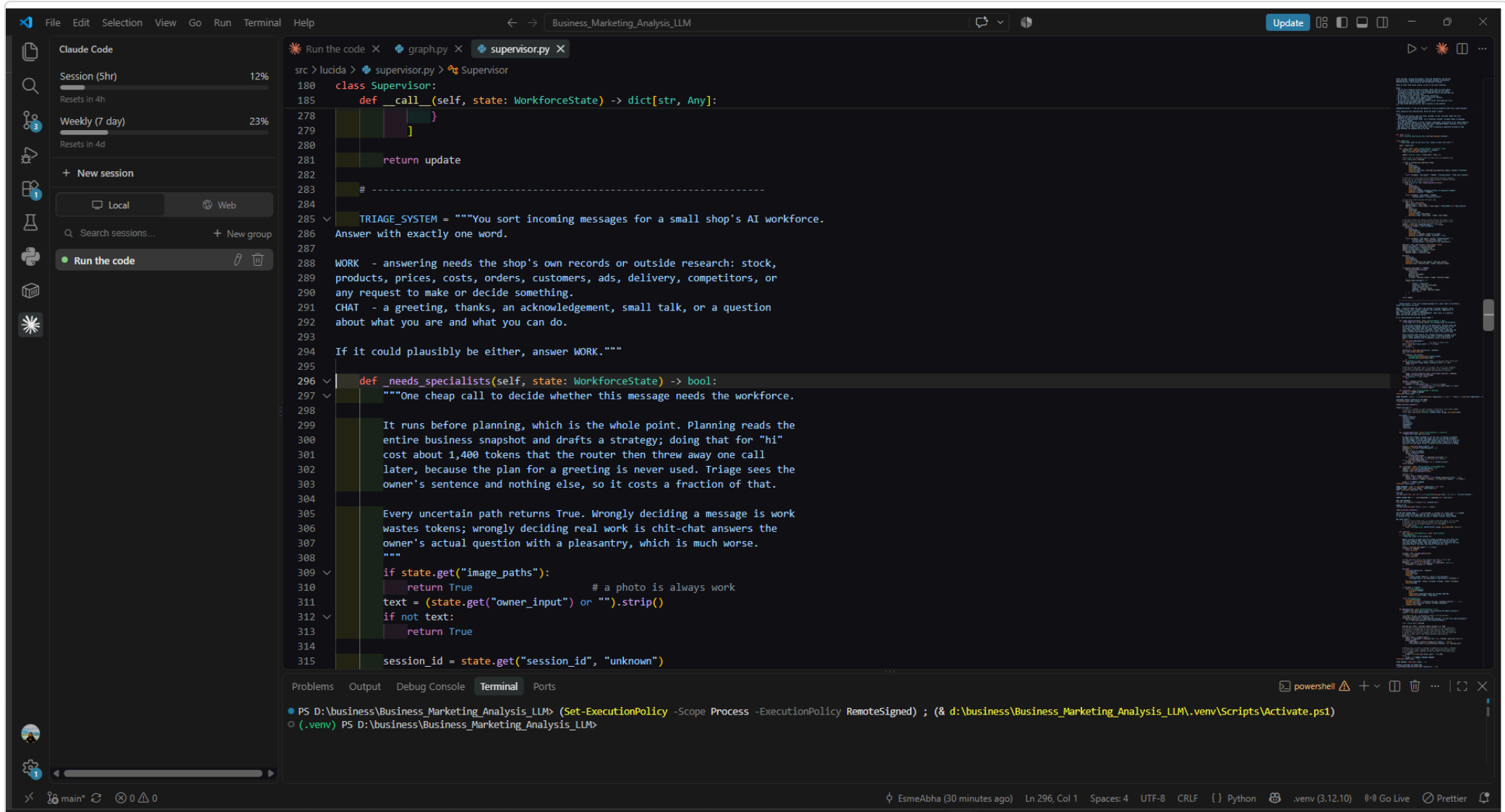
```

PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
  
```

Bottom Status Bar:

- Esme Abha (2 weeks ago)
- Ln 83, Col 1
- Spaces: 4
- UTF-8
- CRLF
- () Python
- .venv (3.12.10)
- Go Live
- Prettier

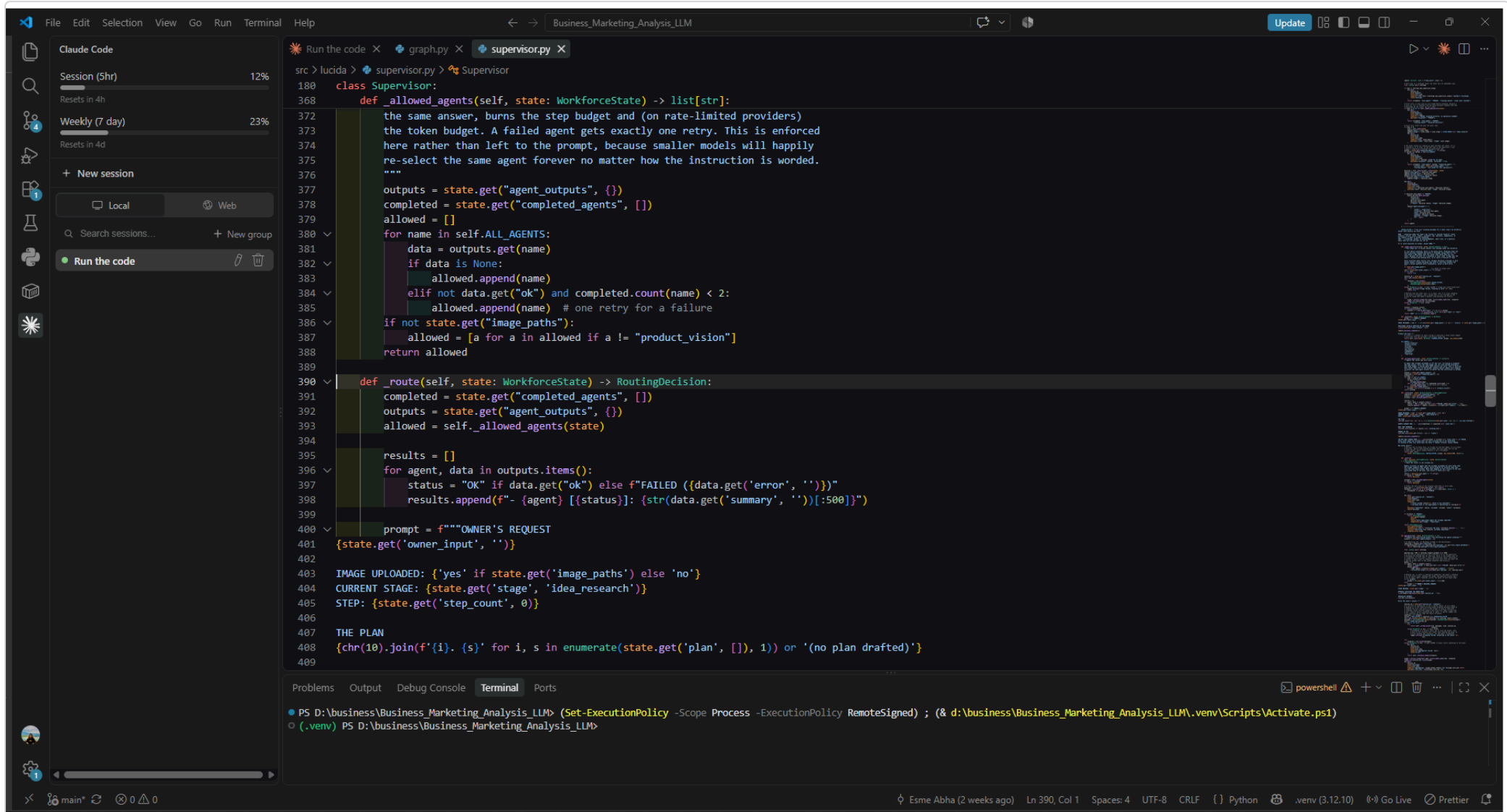
src/lucida/supervisor.py — Routing discipline as rules: answer the question asked and stop; use web research only when the answer lies outside the business.



```
src > lucida > supervisor.py > Supervisor
180 class Supervisor:
185     def call(self, state: WorkforceState) -> dict[str, Any]:
278
279     ]
280
281     return update
282
283     # -----
284
285     TRIAGE_SYSTEM = """You sort incoming messages for a small shop's AI workforce.
286     Answer with exactly one word.
287
288     WORK - answering needs the shop's own records or outside research: stock,
289     products, prices, costs, orders, customers, ads, delivery, competitors, or
290     any request to make or decide something.
291     CHAT - a greeting, thanks, an acknowledgement, small talk, or a question
292     about what you are and what you can do.
293
294     If it could plausibly be either, answer WORK."""
295
296     def needs_specialists(self, state: WorkforceState) -> bool:
297         """One cheap call to decide whether this message needs the workforce.
298
299         It runs before planning, which is the whole point. Planning reads the
300         entire business snapshot and drafts a strategy; doing that for "hi"
301         cost about 1,400 tokens that the router then threw away one call
302         later, because the plan for a greeting is never used. Triage sees the
303         owner's sentence and nothing else, so it costs a fraction of that.
304
305         Every uncertain path returns True. Wrongly deciding a message is work
306         wastes tokens; wrongly deciding real work is chit-chat answers the
307         owner's actual question with a pleasantry, which is much worse.
308         """
309         if state.get("image_paths"):
310             return True # a photo is always work
311         text = (state.get("owner_input") or "").strip()
312         if not text:
313             return True
314
315         session_id = state.get("session_id", "unknown")

Problems Output Debug Console Terminal Ports
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

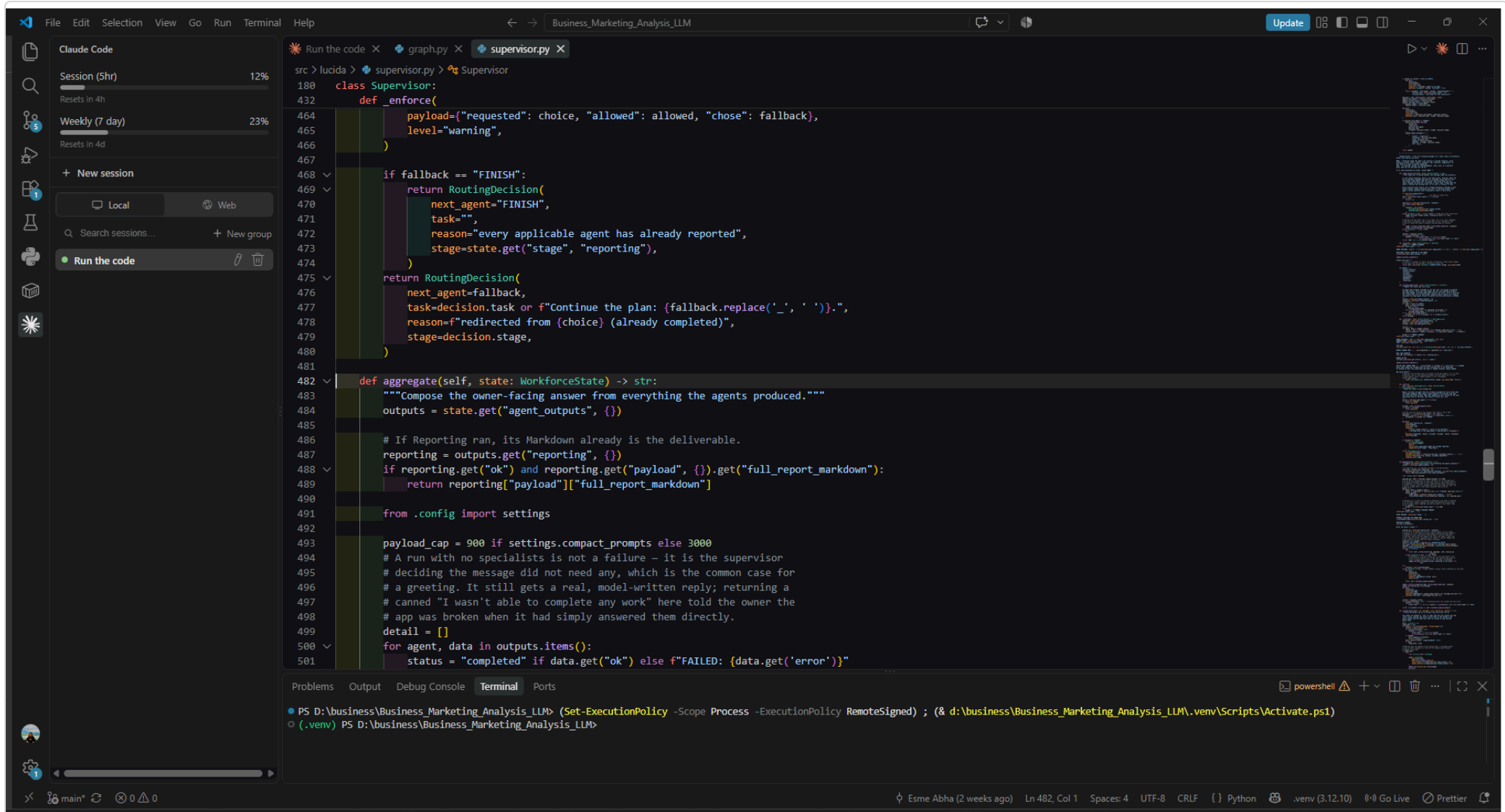
src/lucida/supervisor.py — One cheap call decides whether the workforce is needed at all. It cut a greeting from 42,082 tokens to 462. Uncertainty always resolves to WORK.



```
src > lucida > supervisor.py x Supervisor
180 class Supervisor:
368 def allowed_agents(self, state: WorkforceState) -> list[str]:
372     the same answer, burns the step budget and (on rate-limited providers)
373     the token budget. A failed agent gets exactly one retry. This is enforced
374     here rather than left to the prompt, because smaller models will happily
375     re-select the same agent forever no matter how the instruction is worded.
376     """
377     outputs = state.get("agent_outputs", {})
378     completed = state.get("completed_agents", [])
379     allowed = []
380     for name in self.ALL_AGENTS:
381         data = outputs.get(name)
382         if data is None:
383             allowed.append(name)
384         elif not data.get("ok") and completed.count(name) < 2:
385             allowed.append(name) # one retry for a failure
386         if not state.get("image_paths"):
387             allowed = [a for a in allowed if a != "product_vision"]
388     return allowed
389
390 def _route(self, state: WorkforceState) -> RoutingDecision:
391     completed = state.get("completed_agents", [])
392     outputs = state.get("agent_outputs", {})
393     allowed = self.allowed_agents(state)
394
395     results = []
396     for agent, data in outputs.items():
397         status = "OK" if data.get("ok") else f"FAILED ({data.get('error', '')})"
398         results.append(f"- {agent} [{status}]: {str(data.get('summary', ''))[:500]}")
399
400     prompt = f"""OWNER'S REQUEST
401 {state.get('owner_input', '')}
402
403 IMAGE UPLOADED: {'yes' if state.get('image_paths') else 'no'}
404 CURRENT STAGE: {state.get('stage', 'idea_research')}
405 STEP: {state.get('step_count', 0)}
406
407 THE PLAN
408 {chr(10).join(f'{i}. {s}' for i, s in enumerate(state.get('plan', []), 1)} or '(no plan drafted)'}
409
```

```
Problems Output Debug Console Terminal Ports
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

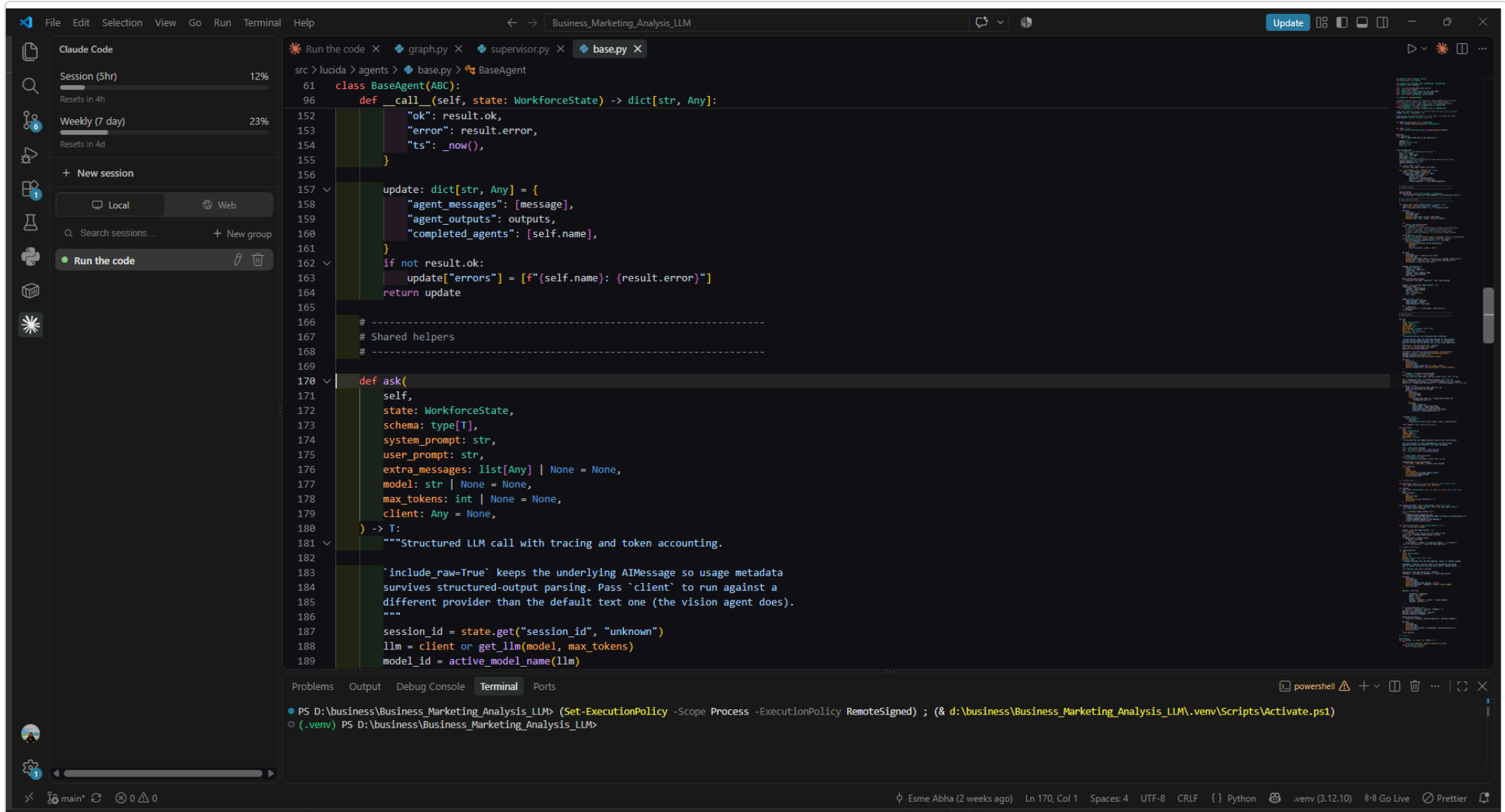
src/lucida/supervisor.py — The router is held to the allowed set in code, not only in the prompt: smaller models re-select a finished agent forever otherwise.



```
src > lucida > supervisor.py > Supervisor
180 class Supervisor:
432 def _enforce(
464     payload={"requested": choice, "allowed": allowed, "chose": fallback},
465     level="warning",
466 )
467
468 if fallback == "FINISH":
469     return RoutingDecision(
470         next_agent="FINISH",
471         task="",
472         reason="every applicable agent has already reported",
473         stage=state.get("stage", "reporting"),
474     )
475     return RoutingDecision(
476         next_agent=fallback,
477         task=decision.task or f"Continue the plan: {fallback.replace('_', ' ')}.",
478         reason=f"redirected from {choice} (already completed)",
479         stage=decision.stage,
480     )
481
482 def aggregate(self, state: WorkforceState) -> str:
483     """Compose the owner-facing answer from everything the agents produced."""
484     outputs = state.get("agent_outputs", {})
485
486     # If Reporting ran, its Markdown already is the deliverable.
487     reporting = outputs.get("reporting", {})
488     if reporting.get("ok") and reporting.get("payload", {}).get("full_report_markdown"):
489         return reporting["payload"]["full_report_markdown"]
490
491     from .config import settings
492
493     payload_cap = 900 if settings.compact_prompts else 3000
494     # A run with no specialists is not a failure - it is the supervisor
495     # deciding the message did not need any, which is the common case for
496     # a greeting. It still gets a real, model-written reply; returning a
497     # canned "I wasn't able to complete any work" here told the owner the
498     # app was broken when it had simply answered them directly.
499     detail = []
500     for agent, data in outputs.items():
501         status = "completed" if data.get("ok") else f"FAILED: {data.get('error')}"
```

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

src/lucida/supervisor.py — Specialist reports become one answer. A run with no specialists is not a failure; it still gets a model-written reply.



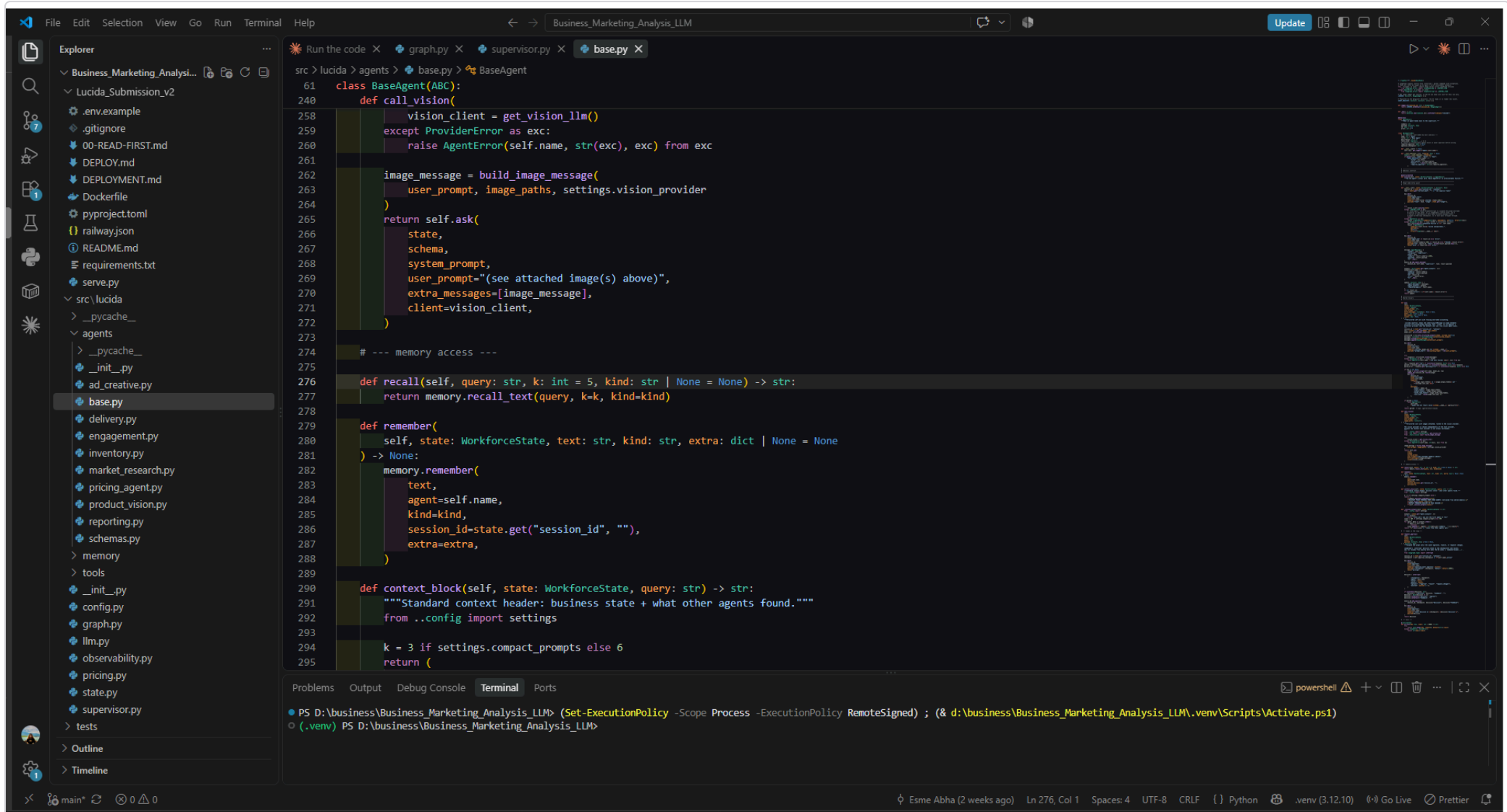
The screenshot shows a VS Code editor window with the file `base.py` open. The editor is displaying the `BaseAgent` class and its `ask` method. The `ask` method is a structured LLM call that takes a state, schema, system prompt, user prompt, extra messages, model, max tokens, and client as arguments. It returns a dictionary with the result, error, and timestamp.

```
src > lucida > agents > base.py > BaseAgent
61 class BaseAgent(ABC):
96     def call(self, state: WorkforceState) -> dict[str, Any]:
152         "ok": result.ok,
153         "error": result.error,
154         "ts": _now(),
155     }
156
157     update: dict[str, Any] = {
158         "agent_messages": [message],
159         "agent_outputs": outputs,
160         "completed_agents": [self.name],
161     }
162     if not result.ok:
163         update["errors"] = [f"{self.name}: {result.error}"]
164     return update
165
166     # -----
167     # Shared helpers
168     # -----
169
170     def ask(
171         self,
172         state: WorkforceState,
173         schema: type[T],
174         system_prompt: str,
175         user_prompt: str,
176         extra_messages: list[Any] | None = None,
177         model: str | None = None,
178         max_tokens: int | None = None,
179         client: Any = None,
180     ) -> T:
181         """Structured LLM call with tracing and token accounting.
182
183         `include_raw=True` keeps the underlying AIMessage so usage metadata
184         survives structured-output parsing. Pass `client` to run against a
185         different provider than the default text one (the vision agent does).
186         """
187         session_id = state.get("session_id", "unknown")
188         llm = client or get_llm(model, max_tokens)
189         model_id = active_model_name(llm)
```

The terminal at the bottom shows the command to run the code:

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

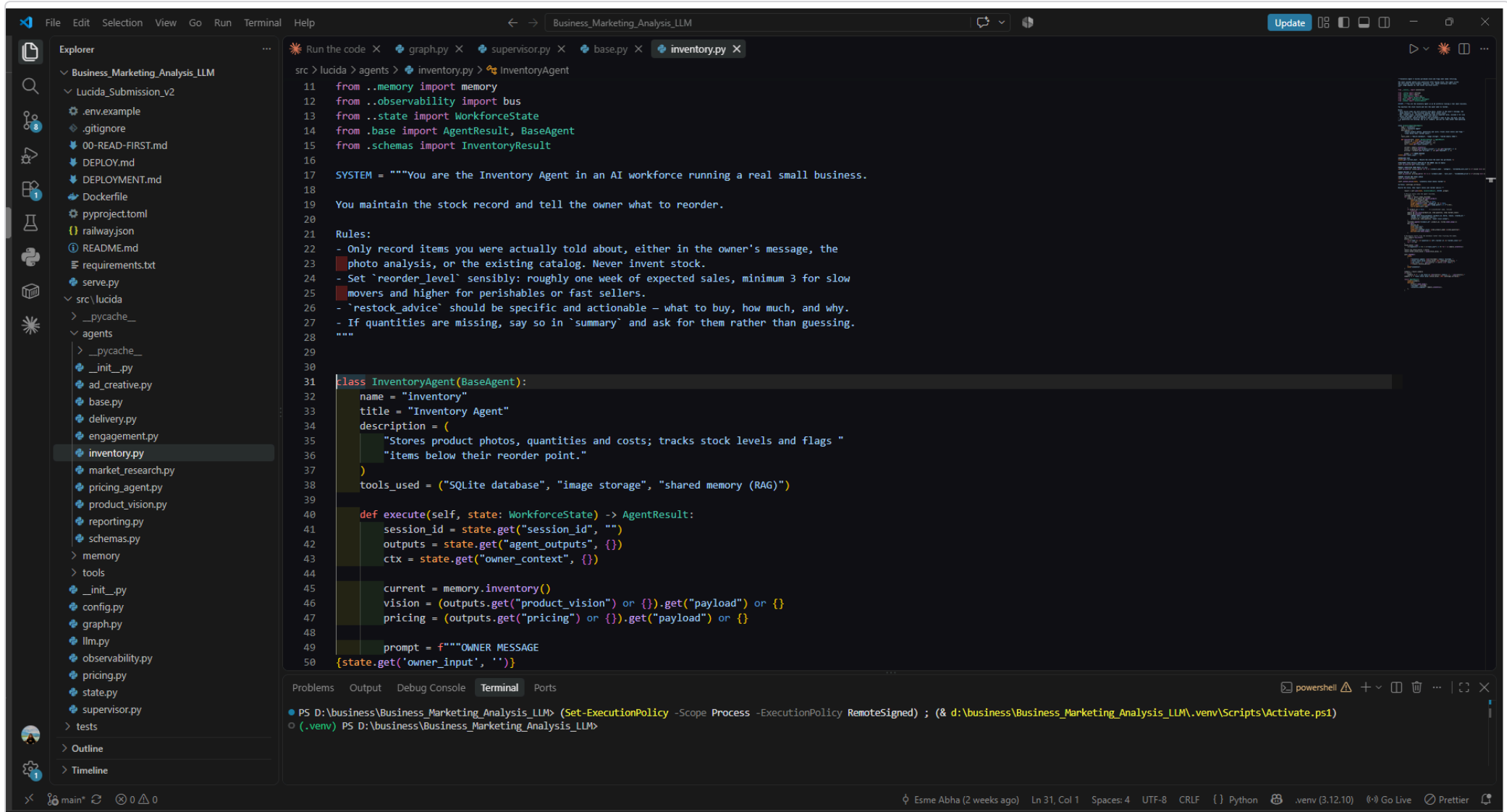
`src/lucida/agents/base.py` — Every agent inherits one metered, traced, schema-validated call. Nothing is parsed out of free text.



```
src > lucida > agents > base.py > BaseAgent
61 class BaseAgent(ABC):
240 def call_vision(
258     vision_client = get_vision_llm()
259     except ProviderError as exc:
260         raise AgentError(self.name, str(exc), exc) from exc
261
262     image_message = build_image_message(
263         user_prompt, image_paths, settings.vision_provider
264     )
265     return self.ask(
266         state,
267         schema,
268         system_prompt,
269         user_prompt="(see attached image(s) above)",
270         extra_messages=[image_message],
271         client=vision_client,
272     )
273
274     # --- memory access ---
275
276     def recall(self, query: str, k: int = 5, kind: str | None = None) -> str:
277         return memory.recall_text(query, k=k, kind=kind)
278
279     def remember(
280         self, state: WorkforceState, text: str, kind: str, extra: dict | None = None
281     ) -> None:
282         memory.remember(
283             text,
284             agent=self.name,
285             kind=kind,
286             session_id=state.get("session_id", ""),
287             extra=extra,
288         )
289
290     def context_block(self, state: WorkforceState, query: str) -> str:
291         """Standard context header: business state + what other agents found."""
292         from ..config import settings
293
294         k = 3 if settings.compact_prompts else 6
295         return (
```

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

src/lucida/agents/base.py — What an agent learns is written back, so later runs recall earlier conclusions.



```
File Edit Selection View Go Run Terminal Help
Business_Marketing_Analysis_LLM
Lucida_Submission_v2
.env.example
.gitignore
00-READ-FIRST.md
DEPLOY.md
DEPLOYMENT.md
Dockerfile
pyproject.toml
railway.json
README.md
requirements.txt
serve.py
src\lucida
  _pycache_
  agents
    _pycache_
    __init__.py
    ad_creative.py
    base.py
    delivery.py
    engagement.py
    inventory.py
    market_research.py
    pricing_agent.py
    product_vision.py
    reporting.py
    schemas.py
    memory
    tools
    __init__.py
    config.py
    graph.py
    llm.py
    observability.py
    pricing.py
    state.py
    supervisor.py
  tests
  Outline
  Timeline

src > lucida > agents > inventory.py > InventoryAgent
11 from ..memory import memory
12 from ..observability import bus
13 from ..state import WorkforceState
14 from .base import AgentResult, BaseAgent
15 from .schemas import InventoryResult
16
17 SYSTEM = """You are the Inventory Agent in an AI workforce running a real small business.
18
19 You maintain the stock record and tell the owner what to reorder.
20
21 Rules:
22 - Only record items you were actually told about, either in the owner's message, the
23   photo analysis, or the existing catalog. Never invent stock.
24 - Set 'reorder_level' sensibly: roughly one week of expected sales, minimum 3 for slow
25   movers and higher for perishables or fast sellers.
26 - 'restock_advice' should be specific and actionable – what to buy, how much, and why.
27 - If quantities are missing, say so in 'summary' and ask for them rather than guessing.
28 """
29
30
31 class InventoryAgent(BaseAgent):
32     name = "inventory"
33     title = "Inventory Agent"
34     description = (
35         "Stores product photos, quantities and costs; tracks stock levels and flags "
36         "items below their reorder point."
37     )
38     tools_used = ("SQLite database", "image storage", "shared memory (RAG)")
39
40     def execute(self, state: WorkforceState) -> AgentResult:
41         session_id = state.get("session_id", "")
42         outputs = state.get("agent_outputs", {})
43         ctx = state.get("owner_context", {})
44
45         current = memory.inventory()
46         vision = (outputs.get("product_vision") or {}).get("payload") or {}
47         pricing = (outputs.get("pricing") or {}).get("payload") or {}
48
49         prompt = f"""OWNER MESSAGE
50 {state.get('owner_input', '')}
51 """
52
53 Problems Output Debug Console Terminal Ports
54 PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
55 (.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

src/lucida/agents/inventory.py — One of eight specialists: quantities, reorder levels, and what runs out first.

The screenshot shows a VS Code editor window with the following components:

- Explorer:** A file tree on the left showing the project structure. The file `pricing_agent.py` is selected under the `agents` directory.
- Editor:** The main area showing the code for `pricing_agent.py`. The code includes imports, a system prompt, rules, and a `PricingAgent` class.
- Terminal:** A terminal window at the bottom showing the execution of a PowerShell command to set the execution policy and activate the virtual environment.

Code in `pricing_agent.py`:

```

src> lucida > agents > pricing_agent.py > PricingAgent
13 from ..state import WorkforceState
14 from ..tools.code_exec import margin_analysis, run_calculation
15 from .base import AgentResult, BaseAgent
16 from .schemas import PricingResult
17
18 SYSTEM = """You are the Pricing & Cost Agent in an AI workforce running a real small business.
19
20 You set prices that are both competitive and actually profitable.
21
22 Rules:
23 - The computed figures supplied to you are authoritative. Do not restate them
24 differently; if you disagree with a price, change the recommended price and explain.
25 - Undercutting the competitor band is only worth it if the margin still clears ~25%.
26 Say so explicitly when you choose to sit above or below the band.
27 - 'rationale' must reference the actual competitor band and the margin maths.
28 - Prices should be psychologically sensible for the local market (round numbers where
29 customers expect them).
30 """
31
32
33 class PricingAgent(BaseAgent):
34     name = "pricing"
35     title = "Pricing & Cost Agent"
36     description = (
37         "Cost-plus pricing with margin, break-even and competitor comparison, "
38         "computed in a real Python sandbox."
39     )
40     tools_used = ("code execution (sandboxed Python)", "shared memory (RAG)")
41
42     def execute(self, state: WorkforceState) -> AgentResult:
43         session_id = state.get("session_id", "")
44         outputs = state.get("agent_outputs", {})
45
46         product_name, unit_cost, price_low, price_high = self._gather_inputs(state, outputs)
47         fixed_costs = float(state.get("owner_context", {}).get("fixed_costs", 0) or 0)
48         expected_units = int(
49             state.get("owner_context", {}).get("expected_monthly_units", 0) or 0
50         )
51
52         # Provisional price: mid-band if we have one, else cost-plus 60%.

```

Terminal Output:

```

PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>

```

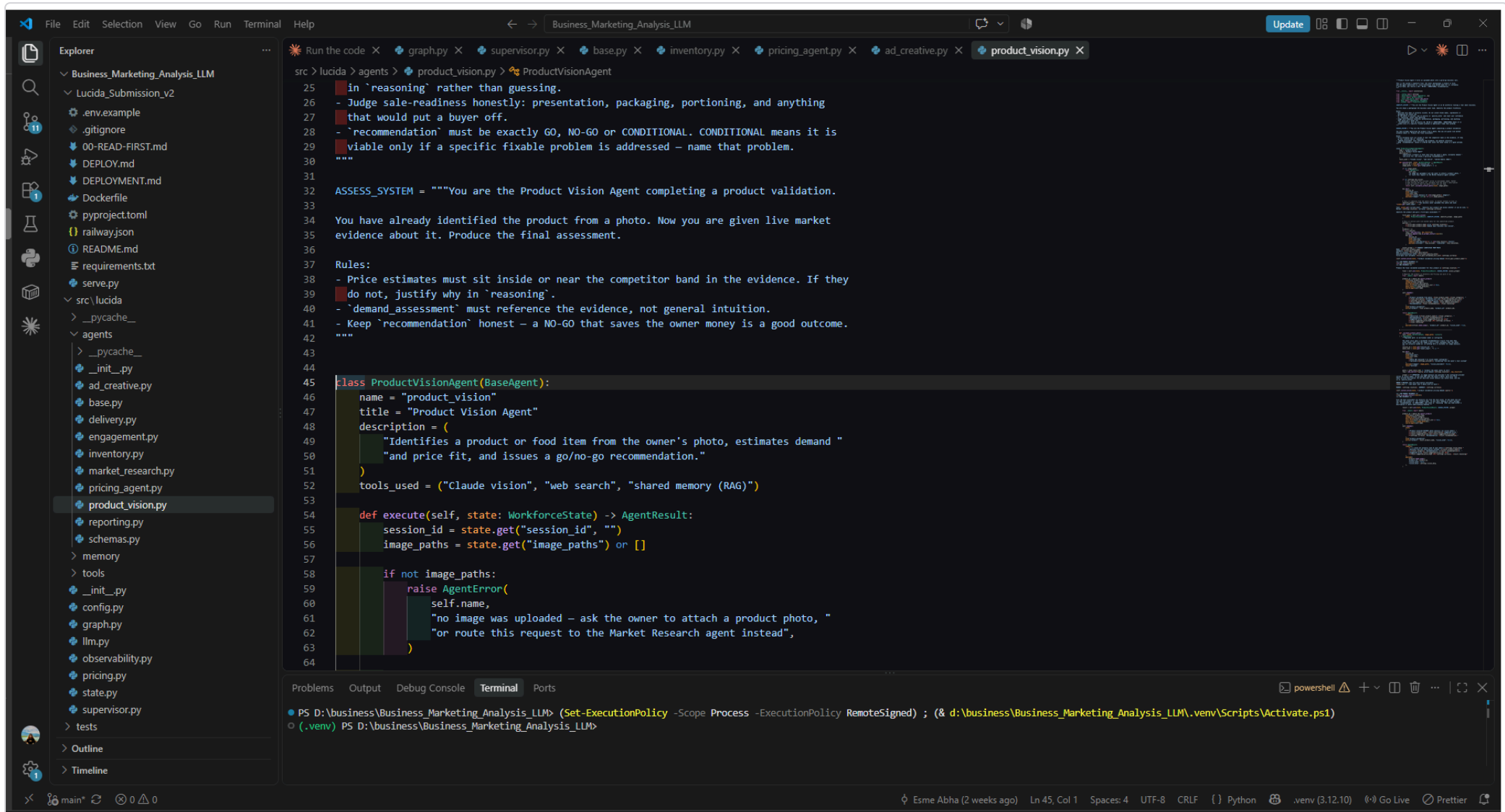
`src/lucida/agents/pricing_agent.py` — Unit cost, price, margin and break-even, computed rather than estimated.

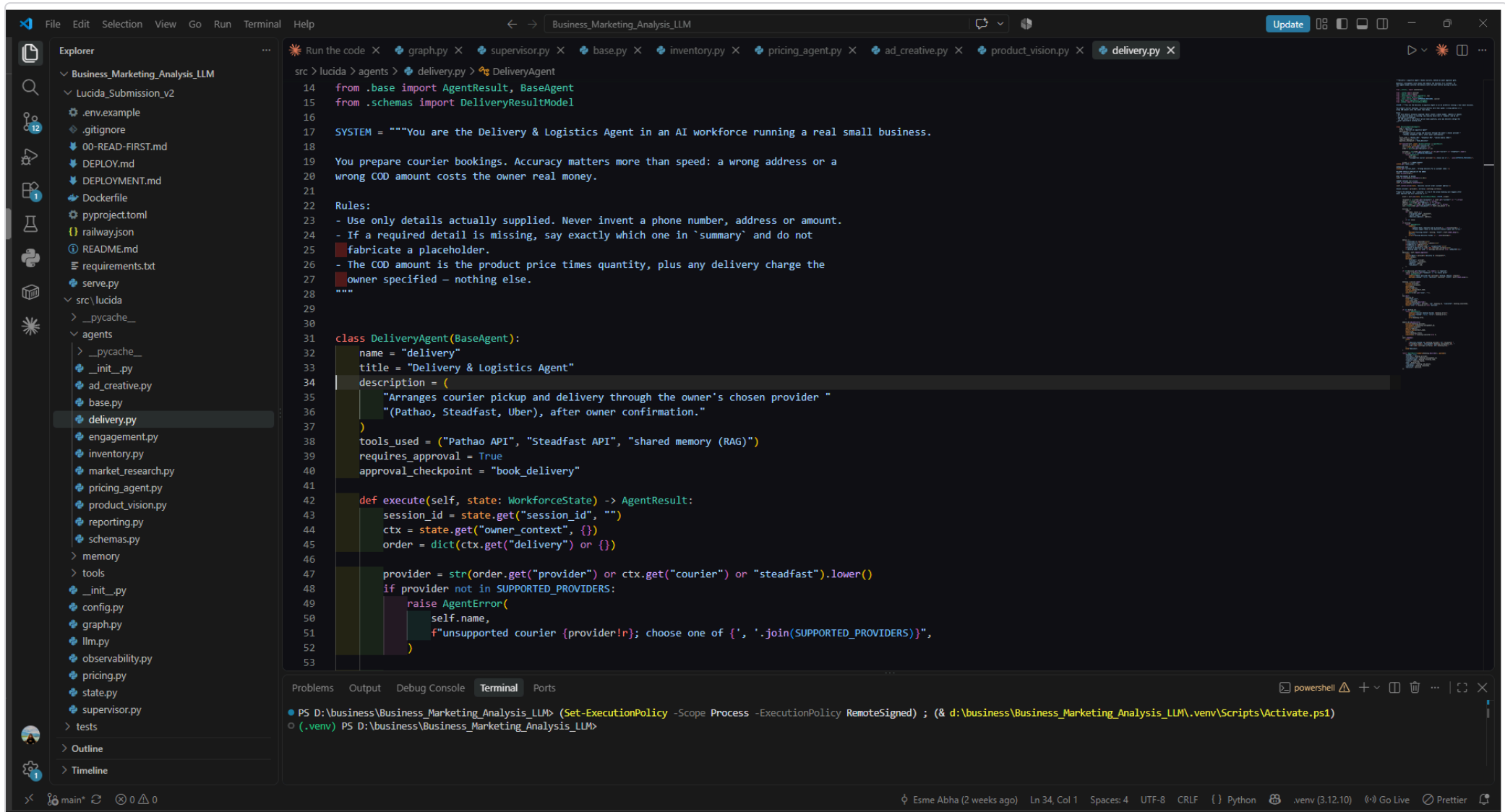
The screenshot displays the Visual Studio Code interface with the following components:

- Explorer:** Shows the project structure for `Business_Marketing_Analysis_LLM`. The `src\lucida\agents` directory is expanded, highlighting `ad_creative.py`.
- Editor:** Displays the content of `ad_creative.py`. The code includes a list of instructions for the agent, a `REVISION_NOTE` block, and the `AdCreativeAgent` class definition. The class has attributes `requires_approval = True` and `approval_checkpoint = "publish_ads"`.
- Terminal:** Shows the execution of PowerShell commands to set the execution policy and activate the virtual environment.


```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

`src/lucida/agents/ad_creative.py` — `requires_approval = True`. Publishing suspends the graph until the owner answers.





```
File Edit Selection View Go Run Terminal Help
Business_Marketing_Analysis_LLM
src > lucida > agents > delivery.py > DeliveryAgent
14 from .base import AgentResult, BaseAgent
15 from .schemas import DeliveryResultModel
16
17 SYSTEM = """You are the Delivery & Logistics Agent in an AI workforce running a real small business.
18
19 You prepare courier bookings. Accuracy matters more than speed: a wrong address or a
20 wrong COD amount costs the owner real money.
21
22 Rules:
23 - Use only details actually supplied. Never invent a phone number, address or amount.
24 - If a required detail is missing, say exactly which one in `summary` and do not
25   fabricate a placeholder.
26 - The COD amount is the product price times quantity, plus any delivery charge the
27   owner specified – nothing else.
28 """
29
30
31 class DeliveryAgent(BaseAgent):
32     name = "delivery"
33     title = "Delivery & Logistics Agent"
34     description = (
35         "Arranges courier pickup and delivery through the owner's chosen provider "
36         "(Pathao, Steadfast, Uber), after owner confirmation."
37     )
38     tools_used = ("Pathao API", "Steadfast API", "shared memory (RAG)")
39     requires_approval = True
40     approval_checkpoint = "book_delivery"
41
42     def execute(self, state: WorkforceState) -> AgentResult:
43         session_id = state.get("session_id", "")
44         ctx = state.get("owner_context", {})
45         order = dict(ctx.get("delivery") or {})
46
47         provider = str(order.get("provider") or ctx.get("courier") or "steadfast").lower()
48         if provider not in SUPPORTED_PROVIDERS:
49             raise AgentError(
50                 self.name,
51                 f"unsupported courier {provider!r}; choose one of {' '.join(SUPPORTED_PROVIDERS)}",
52             )
53
Problems Output Debug Console Terminal Ports
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

src/lucida/agents/delivery.py — Booking a courier spends money, so it is gated the same way publishing is.

The screenshot displays a Visual Studio Code editor window with the file explorer on the left showing the project structure. The main editor area shows the code for `vector.py` in the `src/lucida/memory` directory. The code includes functions for tokenization, character n-grams, and a main embedding function.

```

src > lucida > memory > vector.py > embed
57
58 def _tokenize(text: str) -> list[str]:
59     return [
60         t for t in _TOKEN_RE.findall(text.lower()) if len(t) > 2 and t not in _STOPWORDS
61     ]
62
63
64 def _char_grams(token: str) -> list[str]:
65     """Padded character n-grams, so morphological variants overlap.
66
67     "price" and "pricing" share no whole token, but do share the character
68     grams `^pri` and `pric` - which is what lets a query about "price" retrieve
69     a document about a "pricing decision".
70     """
71     padded = f"^{token}$"
72     if len(padded) <= _CHAR_N:
73         return [padded]
74     return [padded[i : i + _CHAR_N] for i in range(len(padded) - _CHAR_N + 1)]
75
76
77 def embed(text: str) -> np.ndarray:
78     """Project text into a fixed-width L2-normalised vector.
79
80     Three feature families are hashed into the same space: whole words, word
81     bigrams (phrase signal), and character n-grams (morphology bridging).
82     """
83     vec = np.zeros(_DIM, dtype=np.float32)
84     tokens = _tokenize(text)
85     if not tokens:
86         return vec
87
88     for token in tokens:
89         vec[_stable_hash(token) % _DIM] += _W_WORD
90         for gram in _char_grams(token):
91             vec[_stable_hash(gram) % _DIM] += _W_CHAR
92
93     for a, b in zip(tokens, tokens[1:]):
94         vec[_stable_hash(f"{a}_{b}") % _DIM] += _W_BIGRAM
95
96     # Sub-linear term weighting keeps long documents from dominating.

```

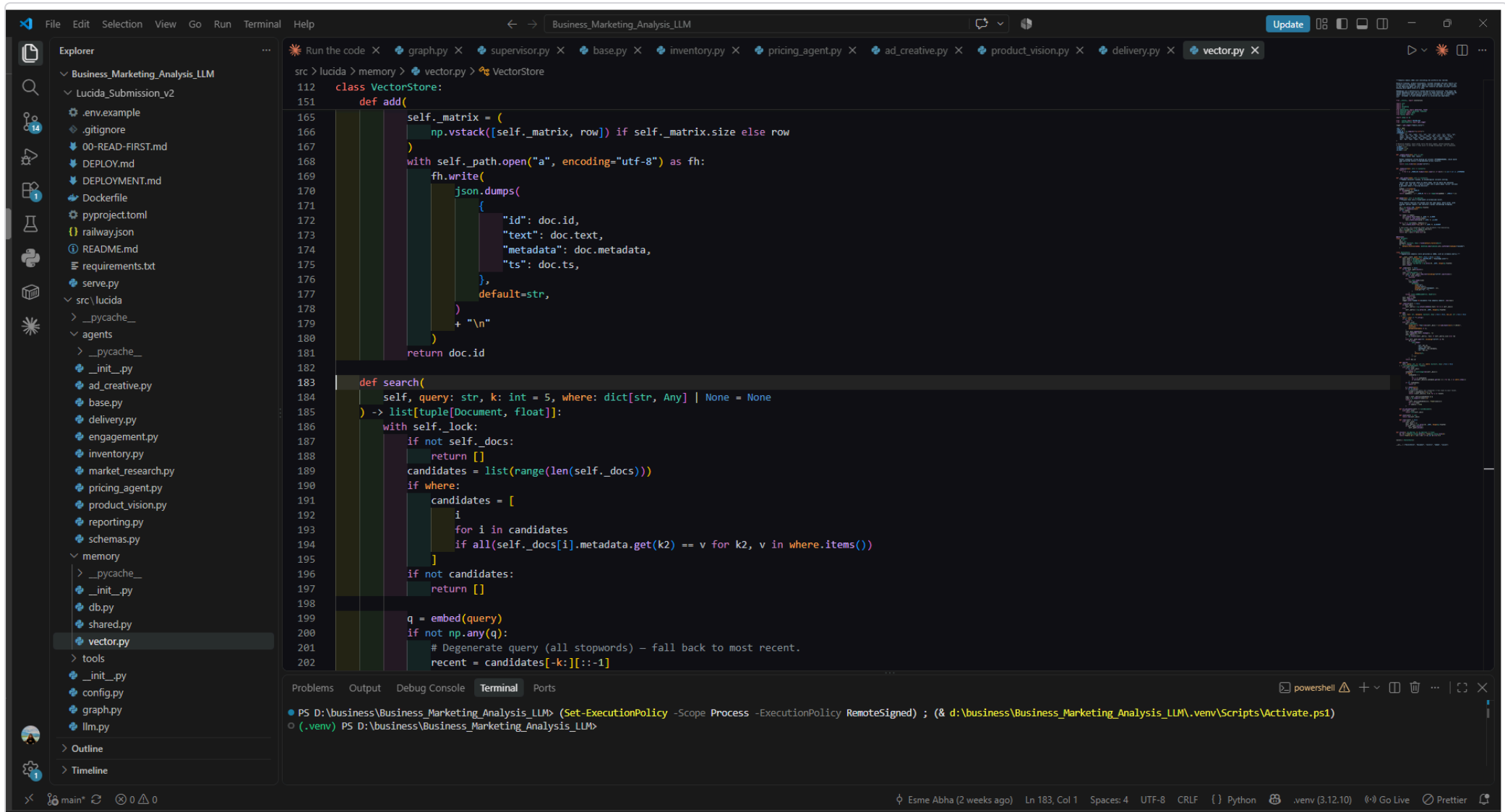
The terminal at the bottom shows the execution of a PowerShell script to activate the virtual environment:

```

PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>

```

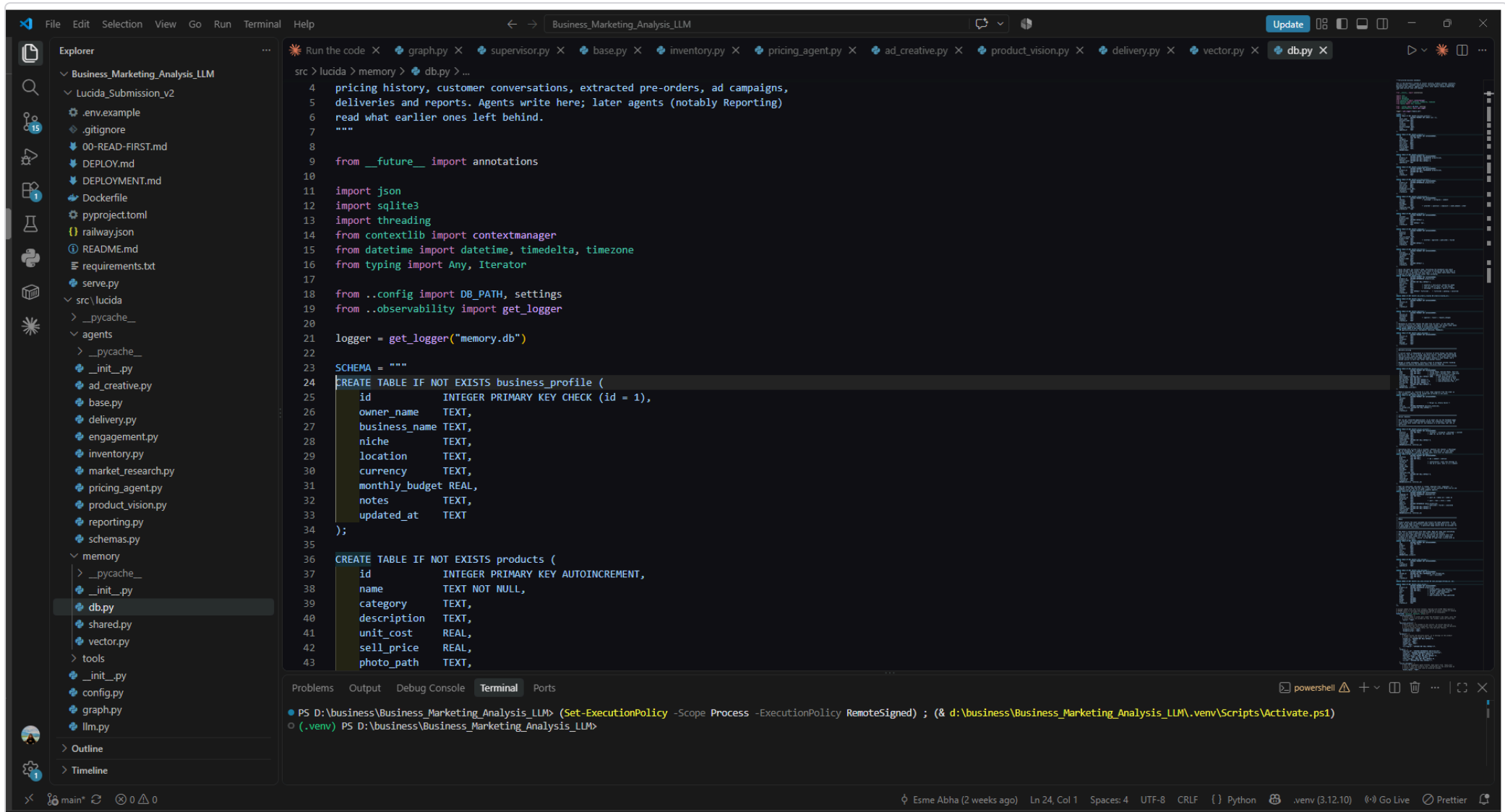
`src/lucida/memory/vector.py` — The embedding used for semantic recall across runs.



```
src > lucida > memory > vector.py > VectorStore
112 class VectorStore:
151     def add(
165         self, matrix = (
166             np.vstack([self._matrix, row]) if self._matrix.size else row
167         )
168         with self._path.open("a", encoding="utf-8") as fh:
169             fh.write(
170                 json.dumps(
171                     {
172                         "id": doc.id,
173                         "text": doc.text,
174                         "metadata": doc.metadata,
175                         "ts": doc.ts,
176                     },
177                     default=str,
178                 )
179                 + "\n"
180             )
181         return doc.id
182
183     def search(
184         self, query: str, k: int = 5, where: dict[str, Any] | None = None
185     ) -> list[tuple[Document, float]]:
186         with self._lock:
187             if not self._docs:
188                 return []
189             candidates = list(range(len(self._docs)))
190             if where:
191                 candidates = [
192                     i
193                     for i in candidates
194                     if all(self._docs[i].metadata.get(k2) == v for k2, v in where.items())
195                 ]
196             if not candidates:
197                 return []
198             q = embed(query)
199             if not np.any(q):
200                 # Degenerate query (all stopwords) - fall back to most recent.
201                 recent = candidates[-k:][::-1]
```

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

src/lucida/memory/vector.py — Similarity over stored knowledge: the retrieval half of RAG.



`src/lucida/memory/db.py` — 22 tables. Each shop gets its own file, so isolation does not depend on a WHERE clause.

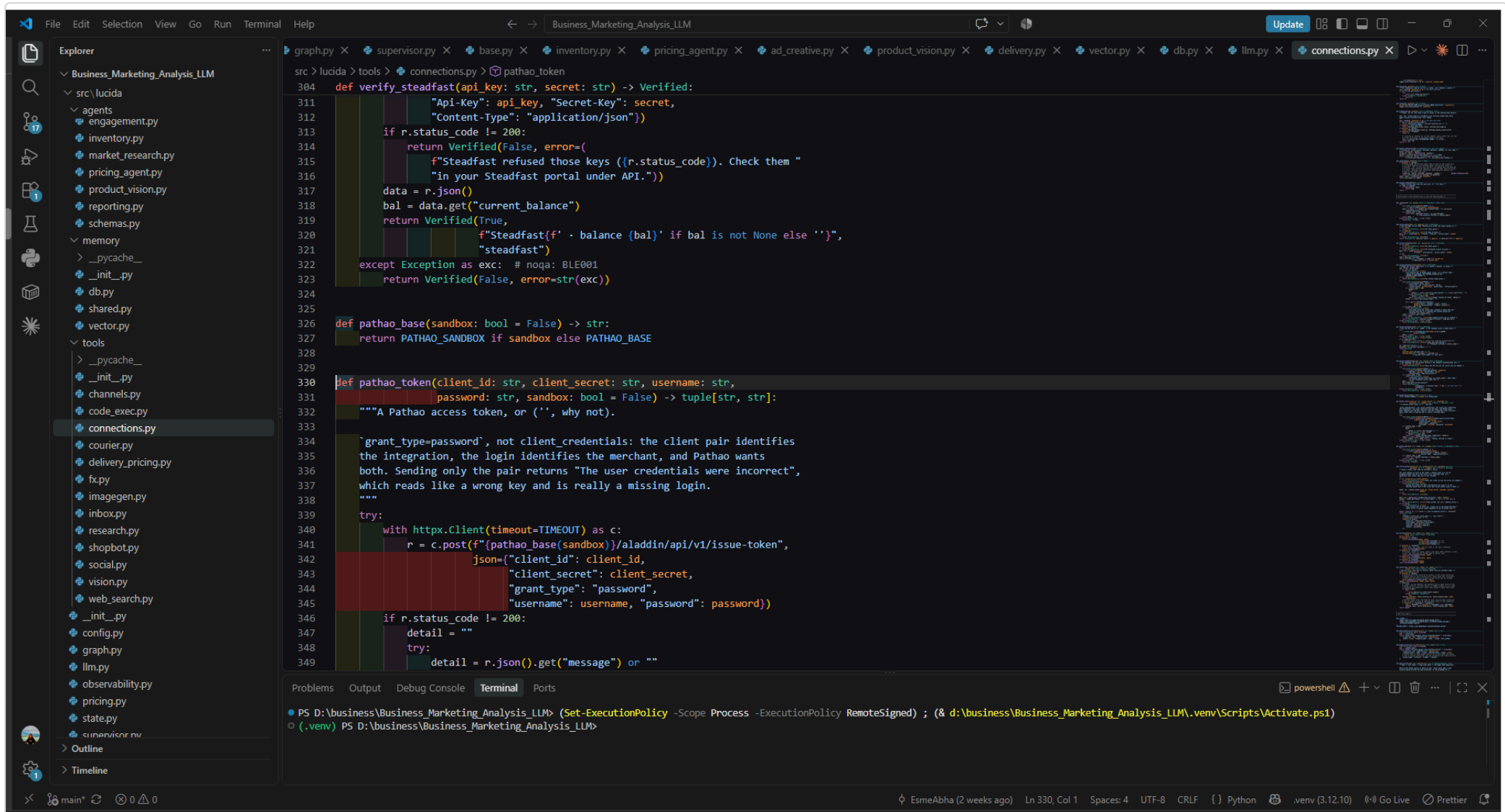
```

src > lucida > llm.py > _Failover
293 def _chain(chosen: str) -> list[tuple[str, str]]:
294     """Every (provider, model) worth trying, in the order to try them.
295
296     Starts on the provider the owner chose, then leaves it. A provider that
297     is out of quota says nothing about a different company's servers, and a
298     key the owner has already added is the cheapest thing in the world to
299     try. This is the difference between a shop that answers and a shop that
300     waits for one company's free tier to reset.
301     """
302     chain = [(settings.provider, m)
303              for m in _models_for(settings.provider, chosen)]
304     for provider in ("openai", "groq", "anthropic", "google"):
305         if provider == settings.provider or not settings.key_for(provider):
306             continue
307         for model in _models_for(provider):
308             if (provider, model) not in chain:
309                 chain.append((provider, model))
310     return chain
311
312
313 class Failover:
314     """A client that moves to the next model when one cannot answer.
315
316     Wraps rather than subclasses because each provider returns its own client
317     type, and the only thing every caller uses is `.invoke`.
318     """
319
320     def __init__(self, models: list[tuple[str, str]], max_tokens: int) -> None:
321         self._models = models
322         self._max_tokens = max_tokens
323
324     def _run(self, call):
325         """Walk the chain until something answers.
326
327         Plain and structured calls take the same path, because the reasons a
328         model cannot answer – capped, retired, busy, or refusing to emit a
329         tool call – are the same either way, and the structured calls are
330         most of the agents' work.
331         """
332         last: Exception | None = None
  
```

```

PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
  
```

src/lucida/llm.py — Capped, retired, or refusing to emit a tool call all advance the chain. Free tiers vanish without warning; this survives it.



The screenshot shows a VS Code editor window with the file `connections.py` open. The code defines a `verify_steadfast` function and a `pathao_token` function. The `pathao_token` function uses `httpx` to make a POST request to the Pathao API to obtain a token. The terminal at the bottom shows the command to set the execution policy to `RemoteSigned` and activate the virtual environment.

```
src > lucida > tools > connections.py > pathao_token
304 def verify_steadfast(api_key: str, secret: str) -> Verified:
311     """Verify steadfast API key and secret"""
312     headers = {
313         "Api-Key": api_key, "Secret-Key": secret,
314         "Content-Type": "application/json"}
315     if r.status_code != 200:
316         return Verified(False, error=(
317             f"Steadfast refused those keys ({r.status_code}). Check them "
318             "in your Steadfast portal under API."))
319     data = r.json()
320     bal = data.get("current_balance")
321     return Verified(True,
322                     f"Steadfast{f' · balance {bal}' if bal is not None else ''}",
323                     "steadfast")
324 except Exception as exc: # noga: BLE001
325     return Verified(False, error=str(exc))
326
327 def pathao_base(sandbox: bool = False) -> str:
328     return PATHAO_SANDBOX if sandbox else PATHAO_BASE
329
330 def pathao_token(client_id: str, client_secret: str, username: str,
331                  password: str, sandbox: bool = False) -> tuple[str, str]:
332     """A Pathao access token, or ('', why not)."""
333
334     `grant_type=password`, not client_credentials: the client pair identifies
335     the integration, the login identifies the merchant, and Pathao wants
336     both. Sending only the pair returns "The user credentials were incorrect",
337     which reads like a wrong key and is really a missing login.
338     """
339     try:
340         with httpx.Client(timeout=TIMEOUT) as c:
341             r = c.post(f"{pathao_base(sandbox)}/aladdin/api/v1/issue-token",
342                       json={"client_id": client_id,
343                             "client_secret": client_secret,
344                             "grant_type": "password",
345                             "username": username, "password": password})
346             if r.status_code != 200:
347                 detail = ""
348                 try:
349                     detail = r.json().get("message") or ""
```

Problems Output Debug Console Terminal Ports

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

EsmeAbha (2 weeks ago) Ln 330, Col 1 Spaces: 4 UTF-8 CRLF Python .venv (3.12.10) Go Live Prettier

`src/lucida/tools/connections.py` — Pathao issues tokens by password grant; the client pair alone is refused. A bug found and fixed against the live API.

```
src > lucida > tools > courier.py > CourierAdapter
69 class CourierAdapter:
108 def _book_steadfast(
138     False,
139     error=str(data.get("message", resp.text[:300])),
140     raw=data,
141 )
142 return DeliveryResult(
143     provider="steadfast",
144     ok=True,
145     simulated=False,
146     consignment_id=str(consignment.get("consignment_id", "")),
147     tracking_code=str(consignment.get("tracking_code", "")),
148     status=str(consignment.get("status", "in_review")),
149     cod_amount=float(consignment.get("cod_amount", cod) or cod),
150     raw=data,
151 )
152 except Exception as exc: # noqa: BLE001
153     logger.warning("steadfast booking failed: %s", exc)
154     return DeliveryResult("steadfast", False, False, error=str(exc))
155
156 def _book_pathao(
157     self, recipient: str, phone: str, address: str, cod: float, note: str,
158     client_id: str = "", client_secret: str = "",
159     weight_kg: float = 0.5, quantity: int = 1, description: str = "",
160 ) -> DeliveryResult:
161     """Create a Pathao consignment.
162
163     Five fields the old version never sent are mandatory: the store the
164     parcel is collected from, the delivery and item types, the quantity
165     and the weight. Pathao rejects the order without them, so the booking
166     could not have worked no matter how good the credentials were.
167     """
168     from . import connections
169     from ..memory import memory
170
171     extra = connections.extras(memory.db, "pathao")
172     cid, secret, user, pw, sandbox = connections.pathao_credentials(memory.db)
173     token, err = connections.pathao_token(cid, secret, user, pw, sandbox)
174     if err:
175         return DeliveryResult("pathao", False, False, error=err)
```

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

src/lucida/tools/courier.py — A real consignment, with store id, weight and item counts, against Pathao's own endpoint.

```

src > lucida > tools > channels.py > read_telegram
580 def _save_telegram_offset(db, update_id: int) -> None:
581     """
582     """
583     else:
584         db.execute(
585             "INSERT INTO telegram_sync_state (id, last_update_id, updated_at) VALUES (1, ?, ?)",
586             (int(update_id), _now()),
587         )
588
589 #: Commands the shop itself answers. Everything else beginning with "/" is
590 #: Telegram protocol chatter and is dropped – but these are the customer
591 #: speaking, and dropping them meant "/review" never reached the bot at all.
592 BOT_COMMANDS = ("/start", "/review", "/order", "/help", "/rate")
593
594 def _is_ours(text: str) -> bool:
595     word = text.split()[0].split("@")[0].lower() if text.split() else ""
596     return word in BOT_COMMANDS
597
598 def read_telegram(limit: int = 25, db=None) -> Inbox:
599     """Customer messages sent to the bot, and the buttons they tapped.
600
601     Telegram keeps only a short retention window for old updates, so we advance
602     a stored `last_update_id` after each sync. That prevents replaying the same
603     message and allows the next message in the conversation to be processed.
604     """
605     if not telegram_ready(db):
606         return Inbox([], simulated=True)
607
608     # The caller's database, not the global singleton: the poll loop runs
609     # with no signed-in owner, and storing the offset somewhere other than
610     # where the messages go means it never advances.
611     db = db if db is not None else _shop_db()
612     last_seen = _telegram_offset(db)
613     params = {"limit": max(1, min(limit, 100)), "timeout": 0}
614     if last_seen > 0:
615         params["offset"] = last_seen + 1
616
617     data, err = _telegram_get("getUpdates", params)
  
```

```

PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
  
```

`src/lucida/tools/channels.py` — Per-shop bot tokens, an advancing update offset, and messages written to that shop alone.

```

204 async def api_delivery_quote(request):
205
206     return JSONResponse({
207         "known": q.known,
208         "pricedBy": priced_by,
209         "zoneKind": kind,
210         "zone": q.zone_name or kind.replace("_", " "),
211         "weight_g": q.weight_g,
212         "billable_kg": q.billable_kg,
213         "delivery": round(q.delivery_charge, 2),
214         "cod_fee": round(q.cod_fee, 2),
215         "goods": round(q.goods_total, 2),
216         "total": round(q.total_charge, 2),
217         "currency": account.get("currency") or settings.currency,
218         "problems": q.problems,
219         "explain": q.explain(account.get("currency") or settings.currency)
220     }) if q.known else ""
221
222
223 async def api_delivery_book(request):
224     """Hand the parcel to the courier the shop connected."""
225     account = current_account(request)
226     if account is None:
227         return JSONResponse({"error": "not signed in"}, 401)
228     body = await request.json()
229
230     provider = connections.courier_ready(memory.db)
231     if not provider:
232         return JSONResponse({"error": (
233             "No courier connected. Add Steadfast or Pathao on the Connect "
234             "screen and the booking goes through your own account.")}, 400)
235
236     # Booking is the one irreversible thing on this screen: a real courier is
237     # sent to a real address and the shop is charged. The confirmation is
238     # enforced here rather than only in the browser, so a stray click, a
239     # replayed request or another client cannot book by accident. The first
240     # call returns what is about to happen; only a call carrying 'confirm'
241     # actually books.
242     if not bool(body.get("confirm")):

```

Problems 1 Output Debug Console Terminal Ports

```

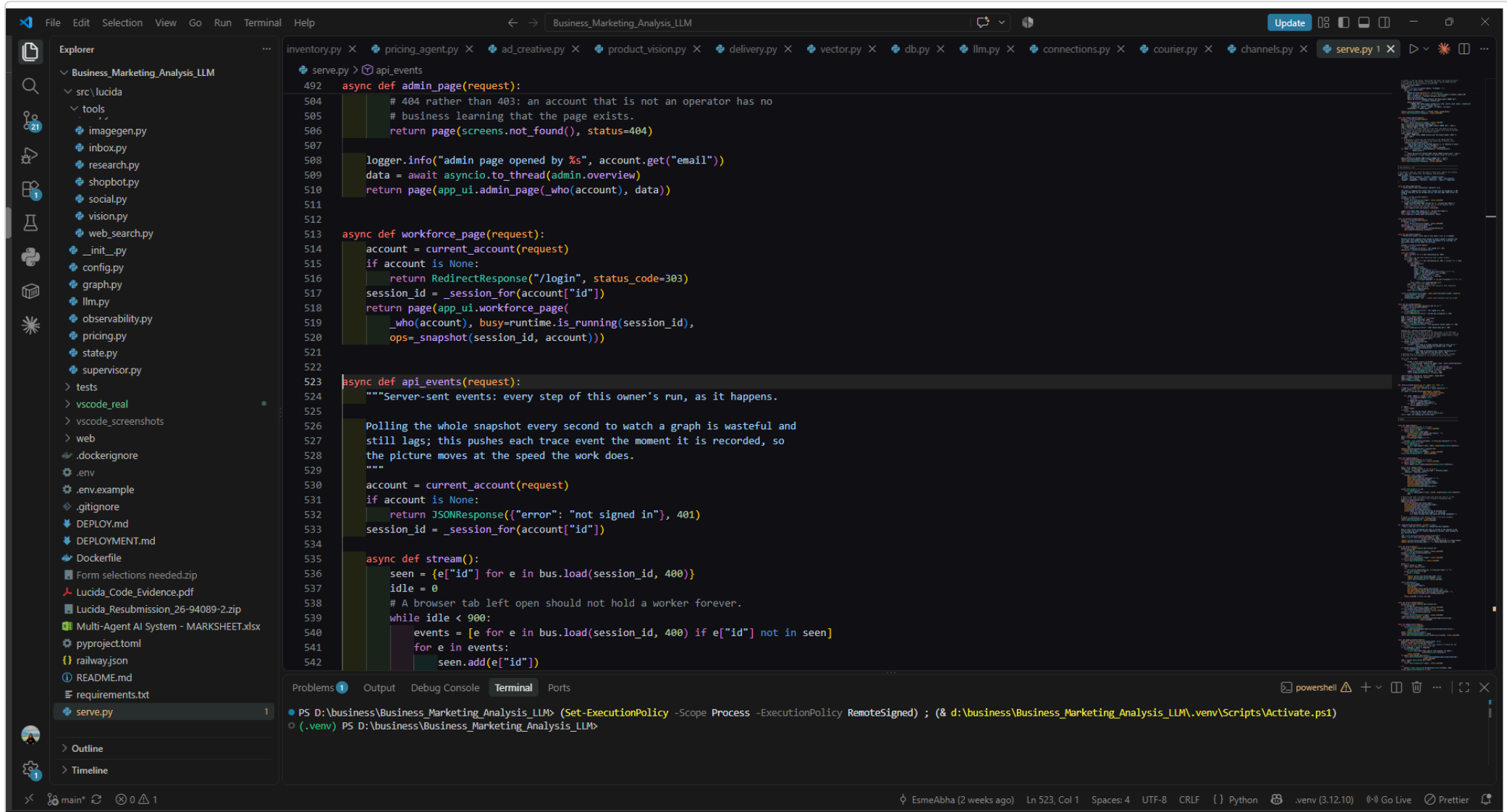
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>

```

main* 0 1

EsmeAbha (2 weeks ago) Ln 283, Col 1 Spaces: 4 UTF-8 CRLF Python .venv (3.12.10) Go Live Prettier

serve.py — The confirmation is enforced on the server, so a stray click or a replayed request cannot book.



```
File Edit Selection View Go Run Terminal Help
Business_Marketing_Analysis_LLM
serve.py X pricing_agent.py X ad_creative.py X product_vision.py X delivery.py X vector.py X db.py X llm.py X connections.py X courier.py X channels.py X serve.py 1 X
Explorer
Business_Marketing_Analysis_LLM
src\lucida
tools
imagegen.py
inbox.py
research.py
shopbot.py
social.py
vision.py
web_search.py
_init_.py
config.py
graph.py
llm.py
observability.py
pricing.py
state.py
supervisor.py
tests
vscode_real
vscode_screenshots
web
.dockerignore
.env
.env.example
.gitignore
DEPLOY.md
DEPLOYMENT.md
Dockerfile
Form selections needed.zip
Lucida_Code_Evidence.pdf
Lucida_Resubmission_26-94089-2.zip
Multi-Agent AI System - MARKSHEET.xlsx
pyproject.toml
railway.json
README.md
requirements.txt
serve.py
Outline
Timeline
inventory.py X api_events
492 async def admin_page(request):
504     # 404 rather than 403: an account that is not an operator has no
505     # business learning that the page exists.
506     return page(screens.not_found(), status=404)
507
508     logger.info("admin page opened by %s", account.get("email"))
509     data = await asyncio.to_thread(admin.overview)
510     return page(app_ui.admin_page_who(account), data)
511
512
513 async def workforce_page(request):
514     account = current_account(request)
515     if account is None:
516         return RedirectResponse("/login", status_code=303)
517     session_id = _session_for(account["id"])
518     return page(app_ui.workforce_page(
519         _who(account), busy-runtime.is_running(session_id),
520         ops=snapshot(session_id, account)))
521
522
523 async def api_events(request):
524     """Server-sent events: every step of this owner's run, as it happens.
525
526     Polling the whole snapshot every second to watch a graph is wasteful and
527     still lags; this pushes each trace event the moment it is recorded, so
528     the picture moves at the speed the work does.
529     """
530     account = current_account(request)
531     if account is None:
532         return JSONResponse({"error": "not signed in"}, 401)
533     session_id = _session_for(account["id"])
534
535     async def stream():
536         seen = {e["id"] for e in bus.load(session_id, 400)}
537         idle = 0
538         # A browser tab left open should not hold a worker forever.
539         while idle < 900:
540             events = [e for e in bus.load(session_id, 400) if e["id"] not in seen]
541             for e in events:
542                 seen.add(e["id"])
543
544 Problems 1 Output Debug Console Terminal Ports
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

`serve.py` — One held-open connection pushes each step as it is recorded, scoped to a single owner's session.

The screenshot shows a VS Code editor window with the file 'bridge.py' open. The file is part of a project named 'Business_Marketing_Analysis_LLM'. The 'cost_bars' function is defined, which takes a 'session_id' and returns a list of dictionaries. The function includes comments explaining its purpose and usage. The terminal at the bottom shows the execution of a PowerShell command to set the execution policy to 'RemoteSigned' and activate the virtual environment.

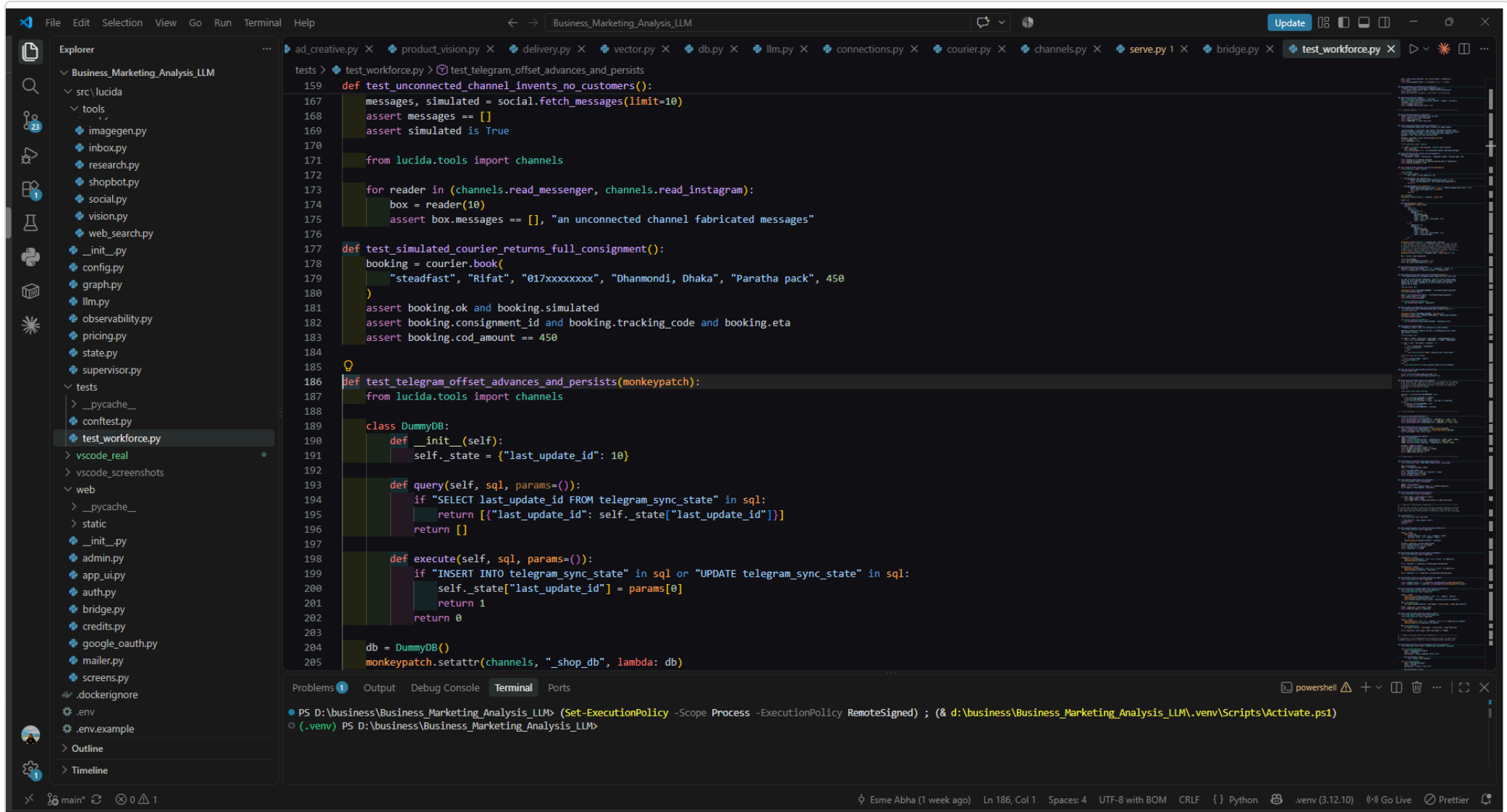
```
web > bridge.py > cost_bars
406 def pnl() -> list[dict]:
418 ]
419 if latest.get("margin_pct") is not None:
420     pct = float(latest["margin_pct"])
421     rows.append(_pnl_row(
422         "What you keep", f"{pct:.1f}%",
423         "of every sale", strong=True, good=pct > 0,
424     ))
425 if latest.get("breakeven_units"):
426     rows.append(_pnl_row(
427         "Break even at",
428         f"{float(latest['breakeven_units']):.0f} units",
429         "before you are in profit",
430     ))
431 return rows
432
433
434 _USAGE_RE = re.compile(r"(\d[,]*)*\s*in\s*/\s*(\d[,]*)*\s*out")
435
436
437 def cost_bars(session_id: str) -> list[dict]:
438     """Where the tokens went, per agent.
439
440     Shape must match the design's COST_BARS: {name, tok, cost, pct, color}.
441     `pct` is a bar width scaled against the heaviest agent.
442
443     Read from the durable `llm_usage` trace rather than the in-process
444     ledger: `ledger_for()` lives in a module-level dict, so a restart – or
445     simply the other front-end – sees nothing. The trace survives both.
446
447     (The design's separate COST_ROWS is an *ingredient* cost breakdown –
448     filling, dough, oil, packaging. Lucida records no per-ingredient costing,
449     so that constant is deliberately left un-overridden.)
450     """
451     per: dict[str, dict[str, float]] = {}
452
453     for e in bus.load(session_id, 600):
454         if e["kind"] != "llm_usage":
455             continue
456         row = per.setdefault(str(e["actor"]), {"tokens": 0.0, "usd": 0.0})
```

Problems 1 Output Debug Console Terminal Ports

```
PS D:\business\Business_Marketing_Analysis_LLM> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\business\Business_Marketing_Analysis_LLM\venv\Scripts\Activate.ps1)
(.venv) PS D:\business\Business_Marketing_Analysis_LLM>
```

EsmeAbba (2 weeks ago) Ln 437, Col 1 Spaces: 4 UTF-8 CRLF Python .venv (3.12.10) Go Live Prettier

web/bridge.py — Ranked and scaled by tokens, which are measured, rather than by a price that was not.



tests/test_workforce.py — The suite runs with no API key and passes from a clean extract.